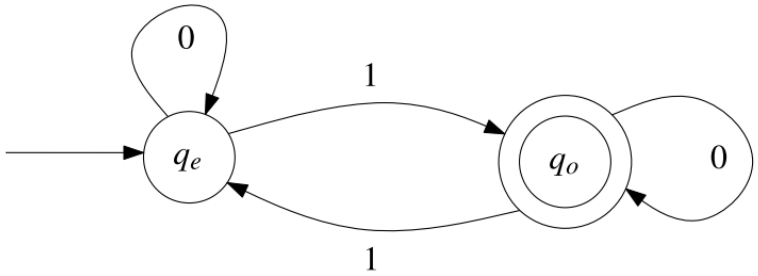
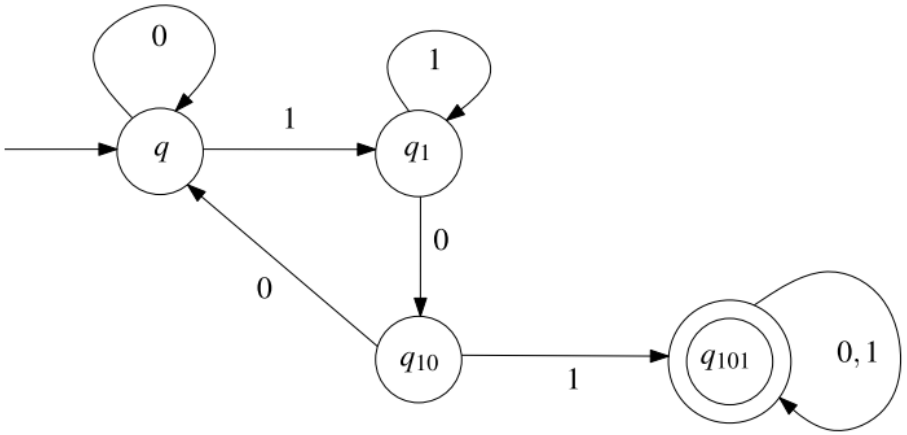
- The Language M: $L(M) = \{w\colon M \text{ accepts } w\}$

  - Language A (set of strings over Σ) is <u>regular</u> if $\exists$ a finite automaton M such that $L(M) = A$.

- Ex #1: Let A = { w : w is a binary string containing an odd number of 1's } .
  - We claim that this language A is regular.
  - In order to prove this, we have to construct a finite automaton M such that A = L(M).



- **Ex #2: Define the language A as: A = { w : w is a binary string containing 101 as a substring }**
  - **We claim that A is regular language.**
  - **In order to prove this, we have to construct a finite automaton M such that A = L(M).**



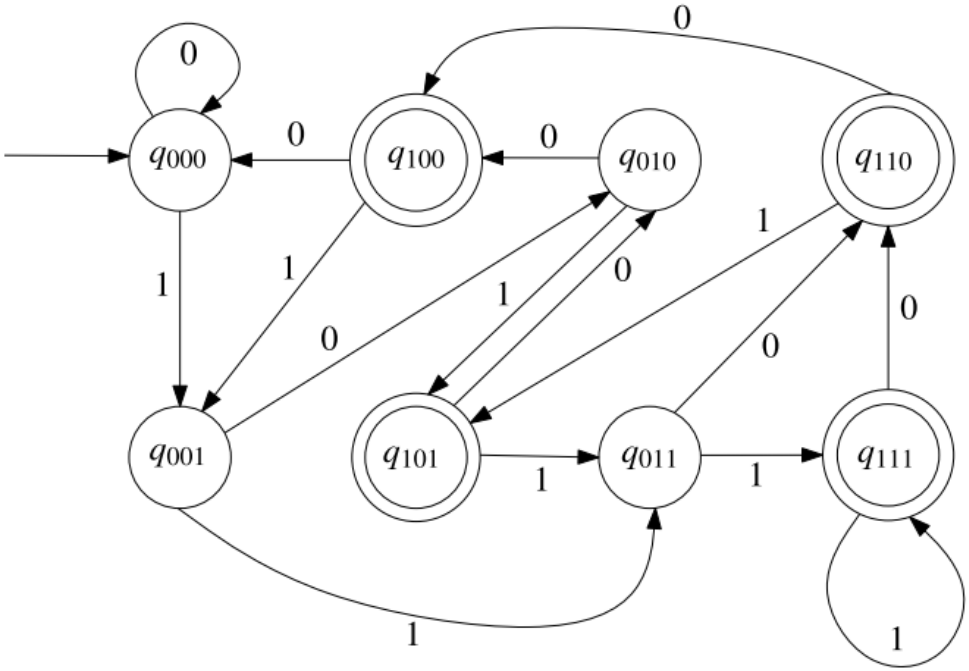
- The finite automata we have seen so far have exactly one accept state. In this example, we will see a finite automaton having more accept states.

- Ex #3: Define the language A as: A = { w : w is a binary string its third bit from the right is 1 }
  - We claim that A is a regular language. To prove this, we have to construct a finite automaton M such that A = L(M).

  - We are looking at a language A made up of all binary strings (strings of 0s and 1s) where the third symbol from the right is a 1.
    - For example:
      - ✓ 110 ∈ A because the third symbol from the right is 1.
      - ✓ 001∉ A because the third symbol from the right is 0.

  - At first sight, it looks be hard to design a finite automaton for this, because the machine has to somehow "look back" and check what the third-to-last symbol was. Finite automata cannot normally count arbitrarily, so how can it do this?

- The Trick: The FA doesn't need to know the whole string. It only needs to remember the last three symbols it has read.
  - If the third-from-last symbol is a 1, then the string is accepted.
  - If it's not, then the string is rejected.



COSC3106: Theory of Computing

Lec#3: Operations on Languages

School of Computer Science and Technology

This lecture

- **In this lecture, you will study…**
  - **The general properties of regular languages and DFA**

Regular operations

- In this lecture, we define operations on languages:
  - Claim #1: If A is a regular language then $\overline{A}$ is regular.
  - Claim #2: If A, B are regular languages then $A \cup B$ is regular.
  - Claim #3: If A, B are regular languages then $A \cap B$ is regular.
  - Claim #4: If A, B are regular languages then the concathenation AB is regular.
  - Claim#5: If A is a regular languages then A^* is regular.

Operations on Languages

- Let A_1 and A_2 be two languages over the same alphabet Σ:
  - Union:

    $$A_1 \cup A_2 = \{w : w \in A_1 \textbf{ OR } w \in A_2\}$$

  - Intersection:

    $$A_1 \cap A_2 = \{w : w \in A_1 \textbf{ AND } w \in A_2\}$$

  - Concatenation:

    $$AB = \{vw \mid v \in A, w \in B\}$$

  - Star:

    $$A^* = \{w_1 w_2 \cdots w_k \mid k \geq 0 \textbf{ and each } w_i \in A\}$$

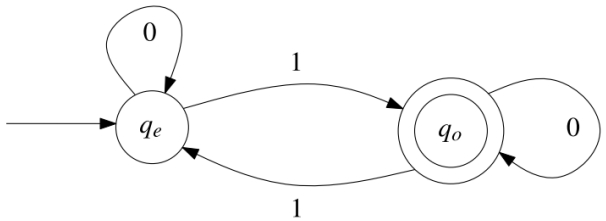
  - Complement:

    $$\overline{A} = \{w \in \Sigma \mid w \notin A\} = \Sigma - A$$

- A finite automaton (FA): $M = (Q, \Sigma, \delta, q, F)$, where
  - Q : finite set of states,
  - Σ : called the alphabet; is a finite set of symbols,
  - $\delta : Q \times \Sigma \rightarrow Q$: Transition function,
  - $q \in Q$: start state,
  - $F \subset Q$: accept states.

Constructing DFA for Complement

- In general, given a DFA M for language A,
- Claim #: If A is a regular language then $\bar{A}$ is regular.
- Why does this work?
  - We can make a DFA $\bar{M}$ for $\bar{A}$ from M by:



M

$\bar{M}$

- Changing all accept states in M into non-accept states in $\bar{M}$,
- changing all non-accept states in M into accept states in $\bar{M}$,

- Assume language A over alphabet Σ has a DFA

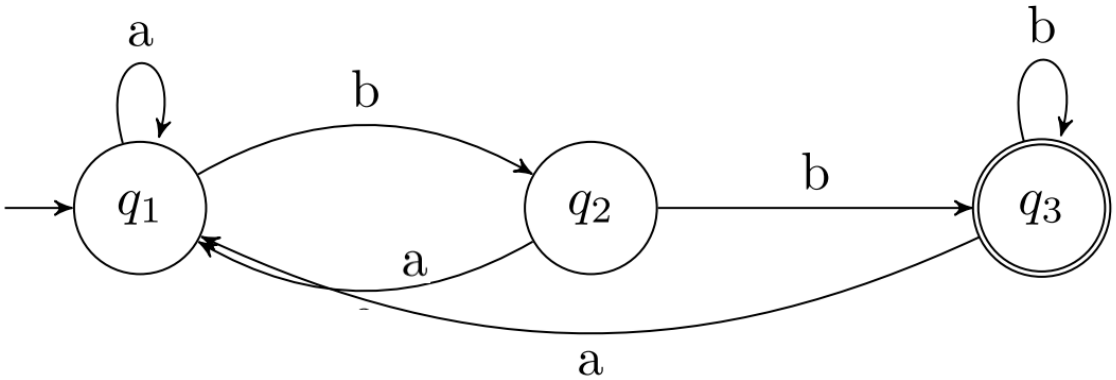
$$M = (Q, \Sigma, \delta, q_1, F).$$

- Then, a DFA for the complementary language $\bar{A}$ is

$$\bar{M} = (Q, \Sigma, \delta, q_1, Q - F).$$

where $Q, \Sigma, \delta, q_1, F$ are the same as in DFA M.

- Ex: Consider the following DFA M with alphabet $\Sigma = \{a, b\}$:



- $L(M)$ is the language of strings over Σ that end with bb, i.e.,

$$L(M) = \{w : w = sbb\}$$

- Note that $abbb \in L(M)$ and $baaa \notin L(M)$

- **Regular Languages Closed Under Union**
  - **Let A_1 and A_2 be two languages over the same alphabet Σ:**
    - **The union of A_1 and A_2 is defined as**

$$A_1 \cup A_2 = \{w : w \in A_1 \; or \; w \in A_2\}$$

    - **Theorem**: *"The set of regular languages is closed under the union operation".*
      - **i.e., if A_1 and A_2 are regular languages over the same alphabet Σ, then $A \cup B$ is also a regular language.**
    - **Proof:**
      - **Suppose A_1 is regular, so it has a DFA M_1.**
      - **Suppose A_2 is regular, so it has a DFA M_2.**
      - **$w \in A_1 \cup A_2$ iff $w \in A_1$ or $w \in A_2$.**
      - **$w \in A_1 \cup A_2$ iff w is accepted by M_1 or M_2.**
      - **Need DFA M_3 to accept a string w iff w is accepted by M_1 or M_2.**
      - **Construct M_3 to keep track of where the input would be if it were simultaneously running on both M_1 and M_2 .**
      - **Accept string iff M_1 or M_2 accepts.**

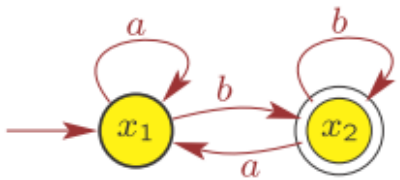
- If $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$,

  then $M_3 = \{Q_3, \Sigma, \delta_3, q_3, F_3\}$ such that:

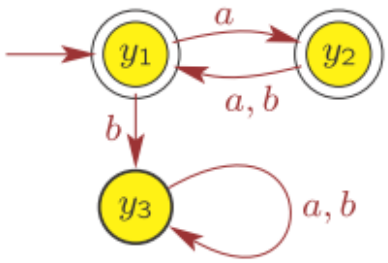
  - $Q_3 = Q_1 \times Q_2 = \{(r_1, r_2): r_1 \in Q_1, r_2 \in Q_2\}$
  - $q_3 = (q_1, q_2)$
  - $F_3 = \{(r_1, r_2): r_1 \in F_1, \boldsymbol{OR}\ r_2 \in F_2\}$
  - $\delta_3\big((r_1, r_2), a\big) = (\delta_1(r_1, a), \delta_2(r_2, a))$

- Ex: Consider the following DFAs and languages over $\Sigma = \{a, b\}$:
  - DFA M_1 accepts language $A_1 = L(M_1)$
  - DFA M_2 accepts language $A_2 = L(M_2)$

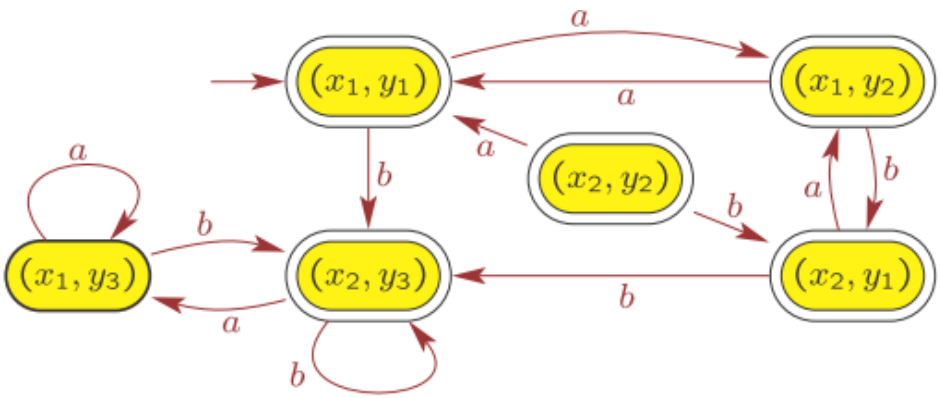
DFA M_1 for A_1



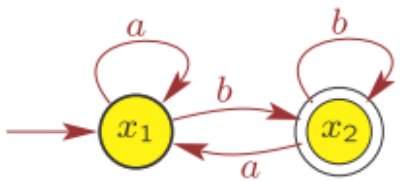
DFA M_2 for A_2



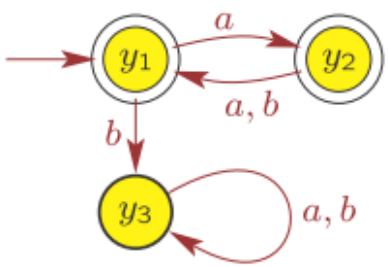
- We now want create a DFA M_3 for $A_1 \cup A_2$.



DFA M_1 for A_1



DFA M_2 for A_2

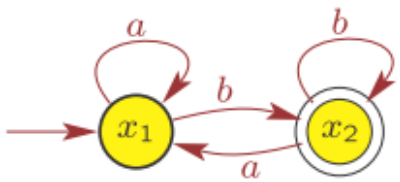


- **We now want a DFA M_3 for $A_1 \cup A_2$.**
  - **Solution**

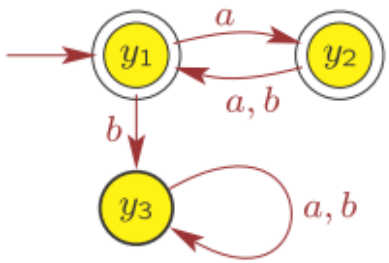
- **Step #1: to build DFA M_3 for $A_1 \cup A_2$: Begin in start states for M_1 and M_2**

- **Step #2: From (x_1, y_1) on input a, M_1 moves to x_1 , and M_2 moves to y_2.**

DFA M_1 for A_1



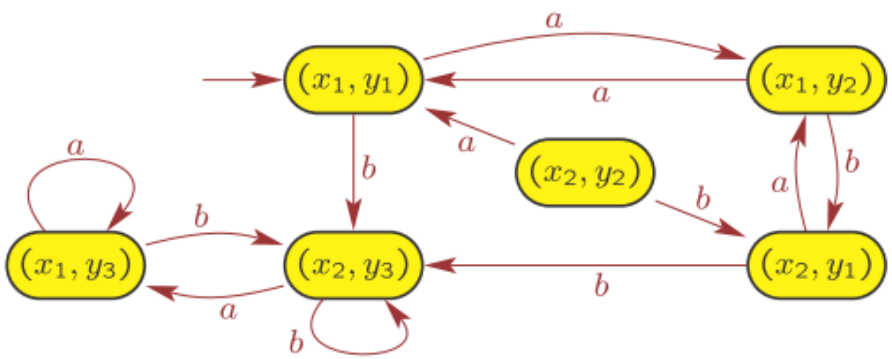
DFA M_2 for A_2



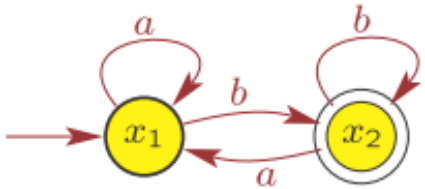
- **We now want a DFA M_3 for $A_1 \cup A_2$.**

Solution

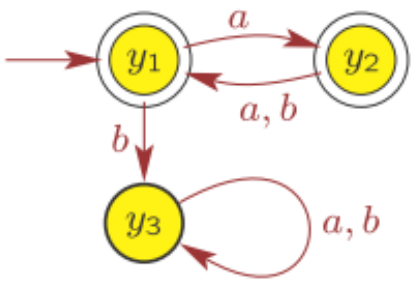
- **Step #3: Continue until each state has outgoing edge for each symbol in Σ.**



DFA M_1 for A_1



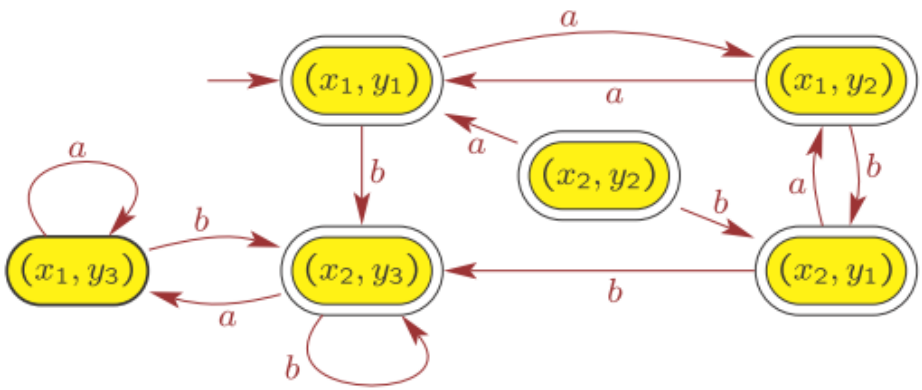
DFA M_2 for A_2



- We now want a DFA M_3 for $A_1 \cup A_2$.

Solution

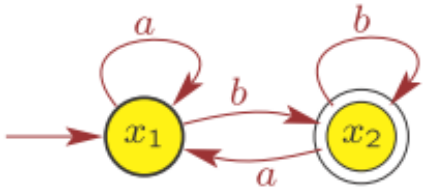
- Step #4: Accept states for DFA M_3 for $A_1 \cup A_2$ have accept state from M_1 and M_2



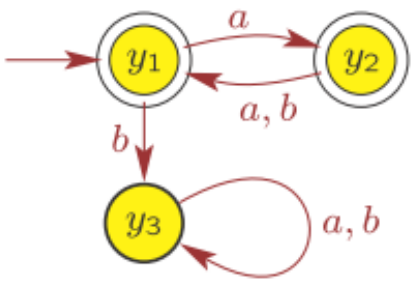
Regular Languages Closed Under Intersection

- **Theorem**: *"The set of regular languages is closed under the intersection operation".*
  - i.e., if A_1 and A_2 are regular languages over the same alphabet Σ, then $A \cap B$ is also a regular language.
- **Proof:**
  - A_1 has a DFA M_1.
  - A_2 has a DFA M_2.
  - $w \in A_1 \cap A_2$ iff $w \in A_1$ and $w \in A_2$.
  - $w \in A_1 \cap A_2$ iff w is accepted by both M_1 and M_2.
  - Need DFA M_3 to accept a string w iff w is accepted by M_1 and M_2.
  - Construct M_3 to keep track of where the input would be if it were simultaneously running on both M_1 and M_2 .
  - Accept string iff M_1 and M_2 accept.

DFA M_1 for A_1



DFA M_2 for A_2

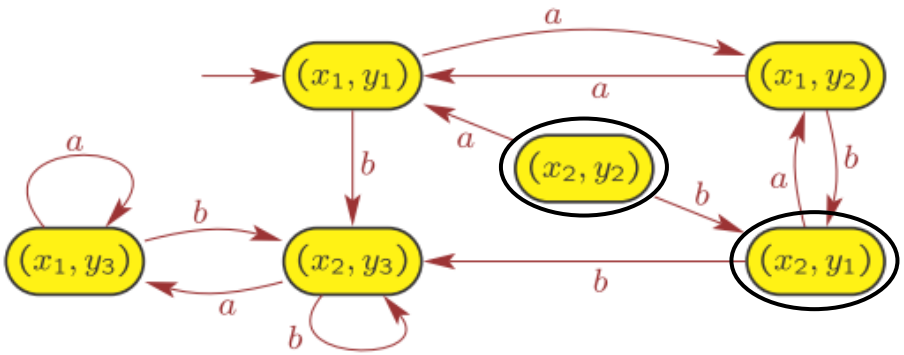


- We now want a DFA M_3 for $A_1 \cap A_2$.

Solution

- Step #4: Accept states for DFA M_3 for $A_1 \cap A_2$ have accept state from M_1 and M_2

DFA M_3 for $A_1 \cap A_2$.



- Concatenation of A and B:

$$AB = \{vw \mid v \in A, w \in B\}$$

  - AB is the set of all strings obtained by taking an arbitrary string v in A and an arbitrary string w in B, and gluing them together (such that v is to the left of w).

- Star of A: A^* is obtained by taking any finite number of strings in A and gluing them together.

  - It is possible (but cumbersome) to directly construct a DFA for AB or for A^* given DFAs for A and B.
  - There is a simpler way if we introduce a new type of machine.